SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

Deep Learning (Unit-02)

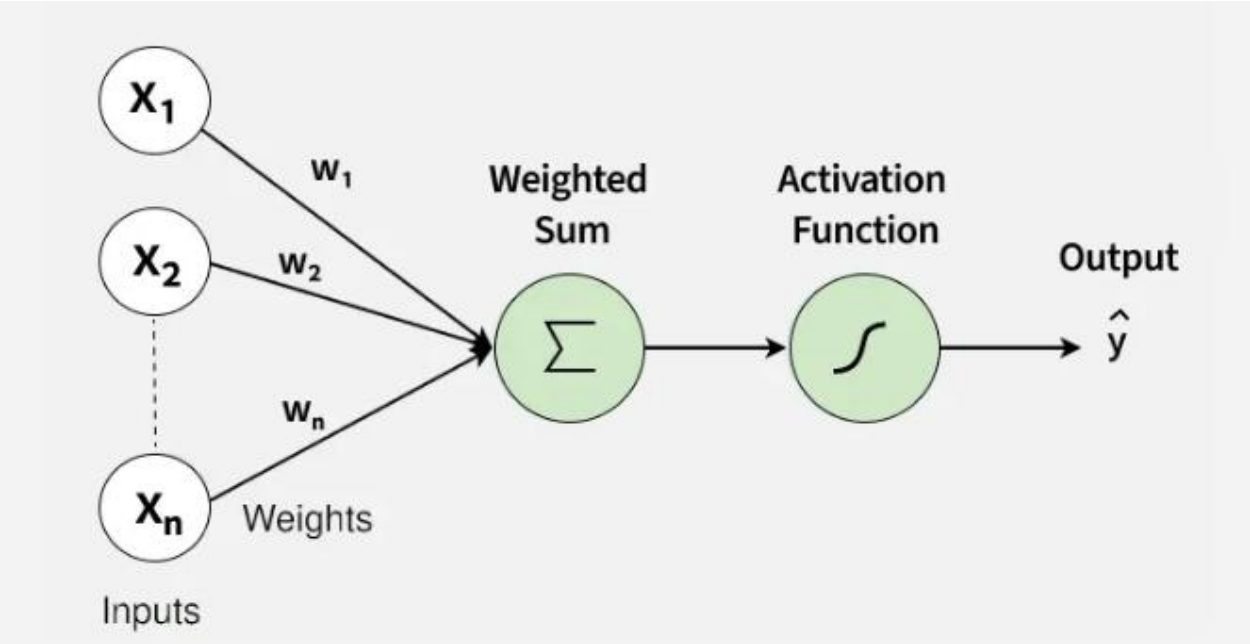
Dr. Zulfikar Ali Ansari

Assistant Professor, Department of AIML

What is Perceptron

A Perceptron is the simplest form of a neural network that makes decisions by combining inputs with weights and applying an activation function. It is mainly used for binary classification problems. It forms the basic building block of many deep learning models.

- Takes multiple inputs and assigns weights
- Computes a weighted sum and applies a threshold
- Outputs either 0 or 1 (binary outcome)
- Forms the foundation of larger neural networks



Core Components

1. Inputs $(x_1, x_2, \ldots, x_n)$

These are the features or measurable attributes of a data point that the perceptron uses to make a decision. Each input provides a signal that contributes to the final output.

- **Example**: For an OR gate, the inputs are binary: $(x_1, x_2) \in \{0, 1\}^2$
- Inputs themselves have no inherent influence unless multiplied by weights.

2. Weights $(w_1, w_2, \ldots, w_n)$

Weights determine how strongly each input contributes to the prediction. A larger weight means the corresponding input has a higher impact.

- *Weights are learned during training, adjusting based on errors.*
- *They act like importance scores for each feature.*

3. Bias (*b*)

The bias is a constant value added to the weighted sum to shift the decision boundary.

- *It allows the perceptron to classify correctly even when all input features are zero.*
- *Bias ensures the model is not forced to pass the decision boundary through the origin.*

Difference Between Weights and Bias

- *Weights control how much each input influences the output.*
- *Bias controls when the perceptron activates, independent of any input.*
- *Mathematically, Weights tilt the line and Bias shifts the line up/down or left/right.*

4. Net Input (Weighted Sum)

This is the combined effect of all inputs and their weights:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

- Represents the activation strength before passing through the activation function.
- If z is high or low enough, it determines the final class.

5. Activation Function (Step Function)

The activation function converts the numerical input into a binary output:

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- It introduces non-linearity in the decision-making, although the decision boundary remains linear.
- Output is always 0 or 1 making perceptrons suitable for binary classification.

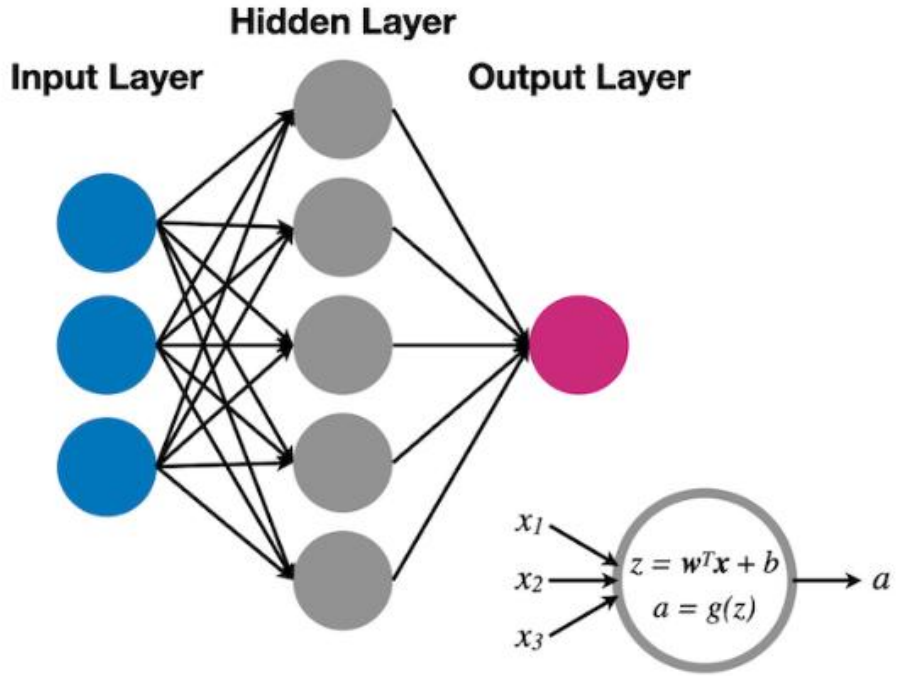
Perceptron vs. Multi-Layer Perceptron (MLP)

Lets compare perceptron and multi-layer perceptron,

Aspect	Perceptron	Multi-Layer Perceptron (MLP)
Model Depth	Single layer with no hidden neurons.	Multiple layers with one or more hidden layers.
Type of Patterns Learned	Learns only linear relationships; straight-line separation.	Learns complex, non-linear patterns and curved boundaries.
Problem-Solving Ability	Cannot solve XOR or non-linearly separable problems.	Easily solves XOR and other complex classification tasks.
Activation Functions	Uses a simple step function (hard 0/1 output).	Uses advanced activations like ReLU, Sigmoid, Tanh for richer learning.
Learning Method	Trained with a simple perceptron update rule.	Trained using backpropagation and gradient descent.
Real-World Use	Limited to simple demonstrations and basic classification.	Used in real-world AI systems like vision, NLP and deep learning applications.

Shallow Neural Networks

A shallow neural network refers to a neural network that consists of only one hidden layer between the input and output layers. This structure is simpler compared to deep neural networks that feature multiple hidden layers. Despite their simplicity, shallow networks are powerful tools capable of approximating any function, given sufficient neurons in the hidden layer a property known as the universal approximation theorem



Components of a Shallow Neural Network

1.Input Layer: This is where the network receives its input data. Each neuron in this layer represents a feature of the input dataset.

2.Hidden Layer: The single hidden layer in a shallow network transforms the inputs into something that the output layer can use. The neurons in this layer apply a set of weights to the inputs and pass them through an activation function to introduce non-linearity to the process.

3.Output Layer: The final layer produces the output of the network. For regression tasks, this might be a single neuron; for classification, it could be multiple neurons corresponding to the classes.

How Do Shallow Neural Networks Work?

The functionality of shallow neural networks hinges on the transformation of inputs through the hidden layer to produce outputs. Here's a step-by-step breakdown:

•**Weighted Sum:** Each neuron in the hidden layer calculates a weighted sum of the inputs.

•**Activation Function:** The weighted sums are passed through an activation function (such as Sigmoid, Tanh, or ReLU) to introduce non-linearity, enabling the network to learn complex patterns.

•**Output Generation:** The output layer integrates the signals from the hidden layer, often through another set of weights, to produce the final output.

Training Shallow Neural Networks

Training a shallow neural network typically involves:

- **Forward Propagation:** Calculating the output for a given input by passing it through the layers of the network.

- **Loss Calculation:** Determining how far the network's output is from the actual desired output using a loss function.

- **Backpropagation:** Calculating the gradient of the loss function with respect to each weight in the network, which informs how the weights should be adjusted to minimize the loss.

- **Weight Update:** Adjusting the weights using an optimization algorithm like gradient descent.

Training Shallow Neural Network for Binary Classification

We will use a synthetic dataset generated using sklearn to illustrate how to build and train this network. Let's create a shallow neural network that will classify data points into two categories.

Step 1: Importing Libraries

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
```

Step 2: Generating and Preprocessing Data

```
# Generate synthetic data
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)

# Split the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Step 3: Building the Shallow Neural Network

```
# Initialize the model
model = Sequential()

# Add the hidden layer with 10 neurons and ReLU activation function
model.add(Dense(10, input_shape=(2,), activation='relu'))

# Add the output layer with sigmoid activation function for binary classification
model.add(Dense(1, activation='sigmoid'))

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.01), loss='binary_crossentropy', metrics=['accuracy'])
```

Step 4: Training the Model

```
# Train the model
history = model.fit(X_train_scaled, y_train, epochs=100, verbose=1, validation_data=(X_test_scaled, y_test))
```

Step 5: Evaluating the Model

```
# Evaluate the model on test data
results = model.evaluate(X_test_scaled, y_test)
print(f"Test Loss: {results[0]}, Test Accuracy: {results[1]}")
```

Training a Perceptron

- *Create a **Perceptron Object***
- *Create a **Training Function***
- ***Train** the perceptron against correct answers*

Create a Perceptron Object

Create a Perceptron object. Name it anything (like Perceptron).

Let the perceptron accept two parameters:

1.The number of inputs (no)

2.The learning rate (learningRate).

Set the default learning rate to 0.00001.

Then create random weights between -1 and 1 for each input.

Training a Perceptron

```
// Perceptron Object
function Perceptron(no, learningRate = 0.00001) {

// Set Initial Values
this.learnc = learningRate;
this.bias = 1;

// Compute Random Weights
this.weights = [];
for (let i = 0; i <= no; i++) {
  this.weights[i] = Math.random() * 2 - 1;
}

// End Perceptron Object
}
```

Training a Perceptron

The Random Weights

*The Perceptron will start with a **random weight** for each input.*

The Learning Rate

For each mistake, while training the Perceptron, the weights will be adjusted with a small fraction.
*This small fraction is the "**Perceptron's learning rate**".*
*In the Perceptron object we call it **learnc**.*

The Bias

- *Sometimes, if both inputs are zero, the perceptron might produce an incorrect output.*
- *To avoid this, we give the perceptron an extra input with the value of 1.*
- *This is called a **bias**.*

Add an Activate Function

Remember the perceptron algorithm:

- Multiply each input with the perceptron's weights
- Sum the results
- Compute the outcome

```
this.activate = function(inputs) {
  let sum = 0;
  for (let i = 0; i < inputs.length; i++) {
    sum += inputs[i] * this.weights[i];
  }
  if (sum > 0) {return 1} else {return 0}
}
```

The activation function will output:

- *1 if the sum is greater than 0*
- *0 if the sum is less than 0*

Create a Training Function

- *The training function guesses the outcome based on the activate function.*
- *Every time the guess is wrong, the perceptron should adjust the weights.*
- *After many guesses and adjustments, the weights will be correct.*

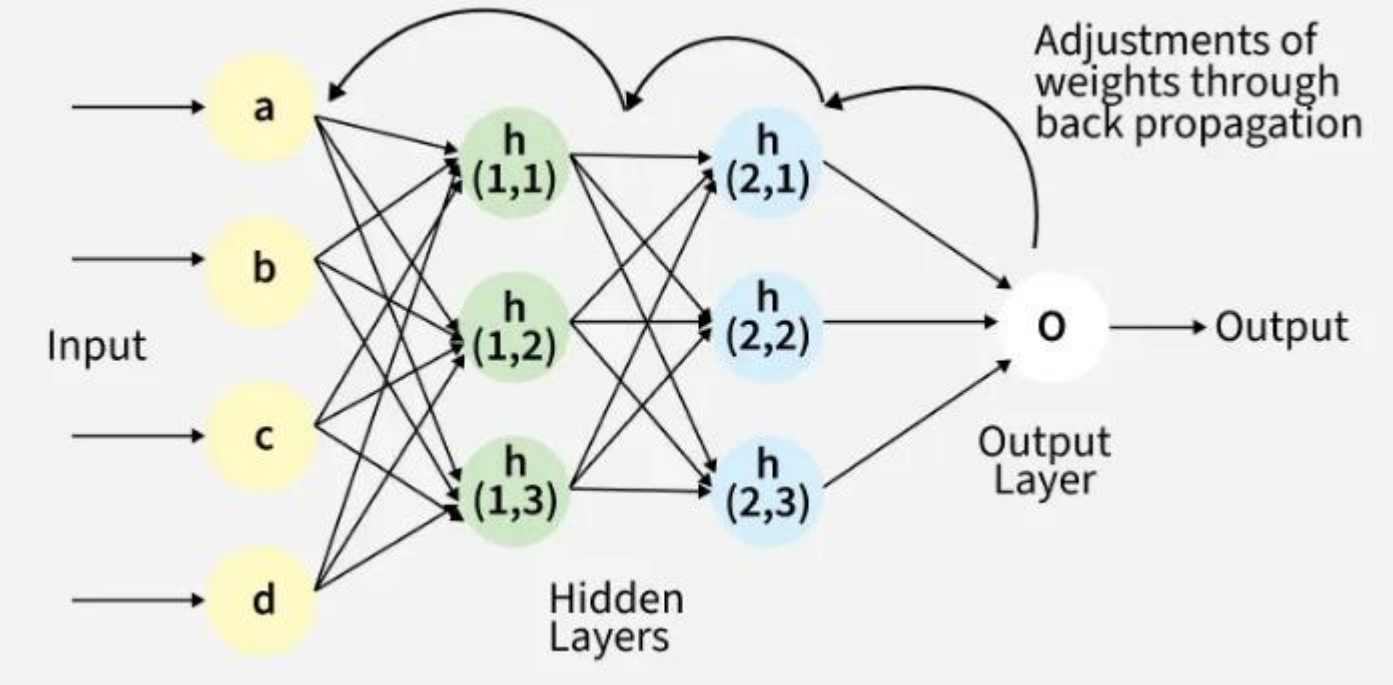
```
this.train = function(inputs, desired) {
  inputs.push(this.bias);
  let guess = this.activate(inputs);
  let error = desired - guess;
  if (error != 0) {
    for (let i = 0; i < inputs.length; i++) {
      this.weights[i] += this.learnc * error *
inputs[i];
    }
  }
}
```

Backpropagation in Neural Network

Backpropagation in Neural Network

Backpropagation is an algorithm that trains neural networks by reducing prediction error. It works by propagating errors backward, computing gradients using the chain rule, and updating weights and biases to improve performance.

- *Computes gradients of the loss function with respect to each weight using the chain rule*
- *Updates weights and biases efficiently to reduce error*
- *Scales well to deep and complex neural networks*
- *Enables automatic learning and continuous improvement across training iterations*



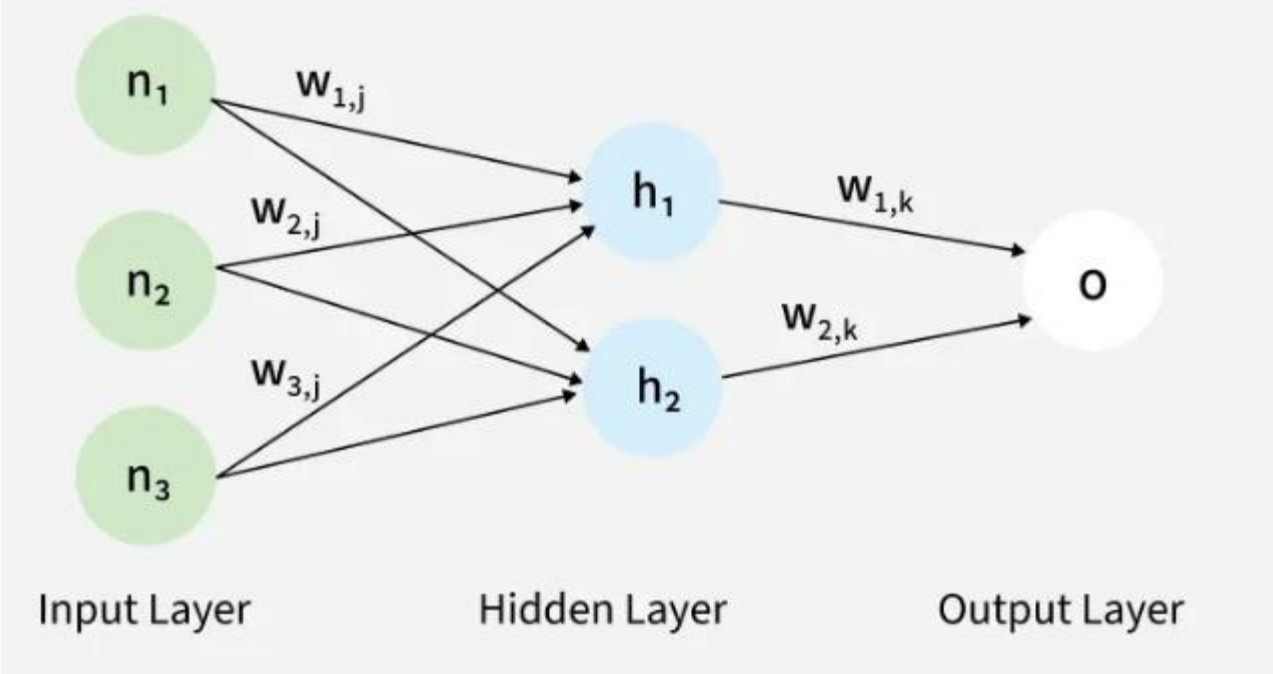
Working of Back Propagation Algorithm

The Back Propagation algorithm involves two main steps:

1. Forward Pass Work

In forward pass, input data moves through the network to generate an output.

- *Input data is passed to the input layer and forwarded to hidden layers*
- *Each neuron computes a weighted sum and adds a bias*
- *Activation functions (like ReLU) are applied to produce outputs*
- *Outputs from one layer become inputs to the next layer*
- *Final layer uses functions like softmax to produce predictions (e.g., probabilities for classification)*



2. Backward Pass

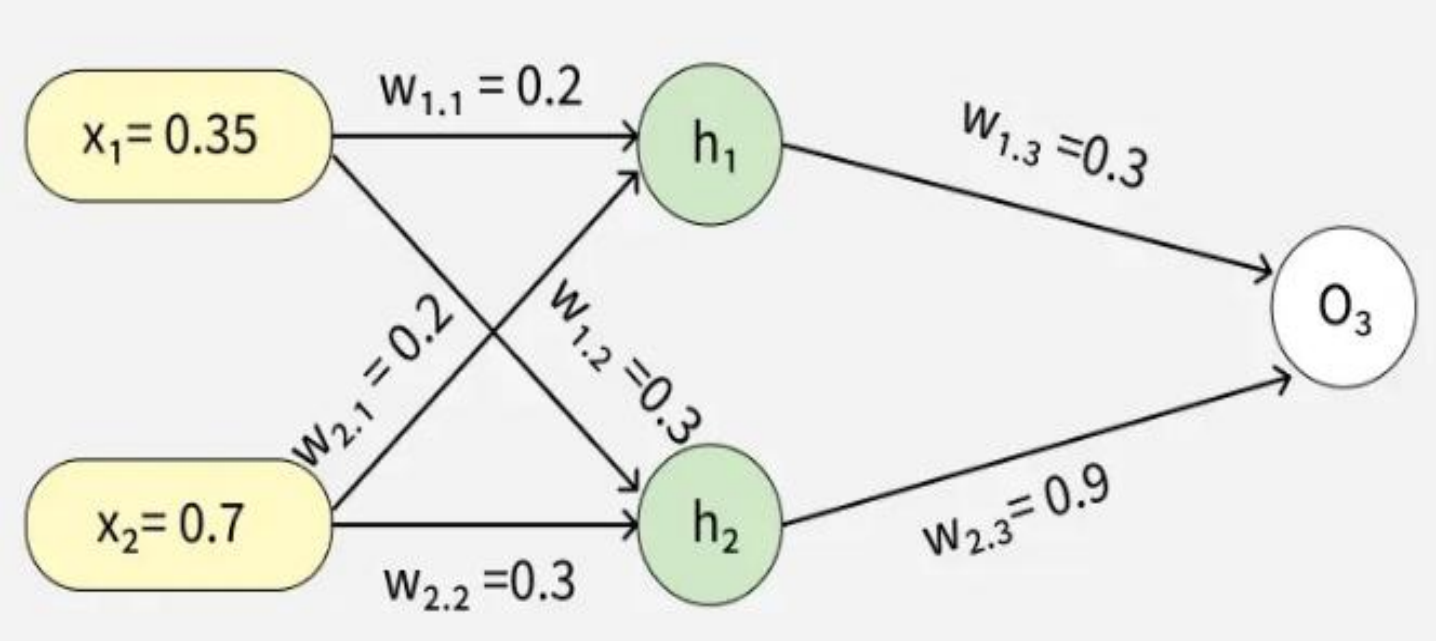
In this step, the error between predicted and actual output is propagated backward to update weights and biases. Error is calculated (e.g., using Mean Squared Error)

$$MSE=(\text{Predicted Output}-\text{Actual Output})2$$

- *Gradients are computed using the chain rule*
- *Weights and biases are updated to reduce the error*
- *Process continues layer by layer from output to input*
- *Derivatives of activation functions are used to compute gradients effectively*

Example

Let's walk through an example of Back Propagation in machine learning. Assume the neurons use the sigmoid activation function for the forward and backward pass. The target output is 0.5 and the learning rate is 1.



Forward Propagation

1. Initial Calculation

The weighted sum at each node is calculated using:

$$aj = \sum(wi, j * xi)$$

Where,

- a_j is the weighted sum of all the inputs and weights at each node
- $w_{i,j}$ represents the weights between the i^{th}input and the j^{th} neuron
- x_i represents the value of the i^{th} input

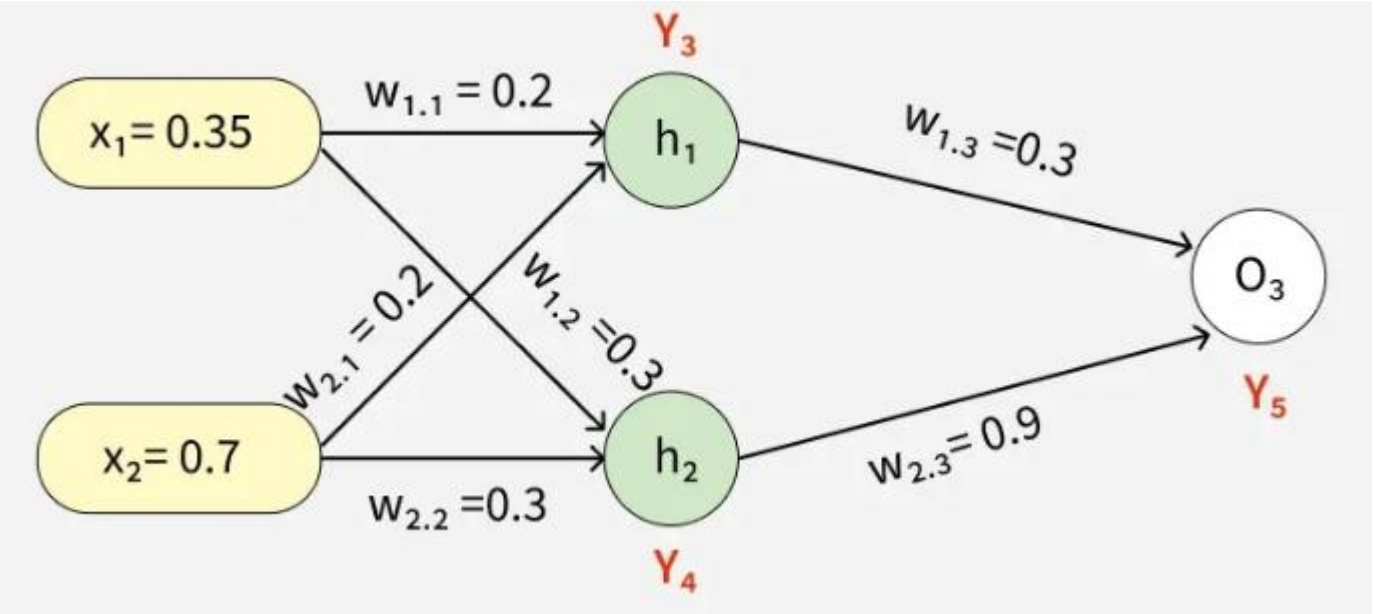
O (output): After applying the activation function to a, we get the output of the neuron:

$$o_j = activation\ function(a_j)$$

2. Sigmoid Function

The sigmoid function returns a value between 0 and 1, introducing non-linearity into the model.

$$y_j = \frac{1}{1+e^{-a_j}}$$



3. Computing Outputs

At h1 node

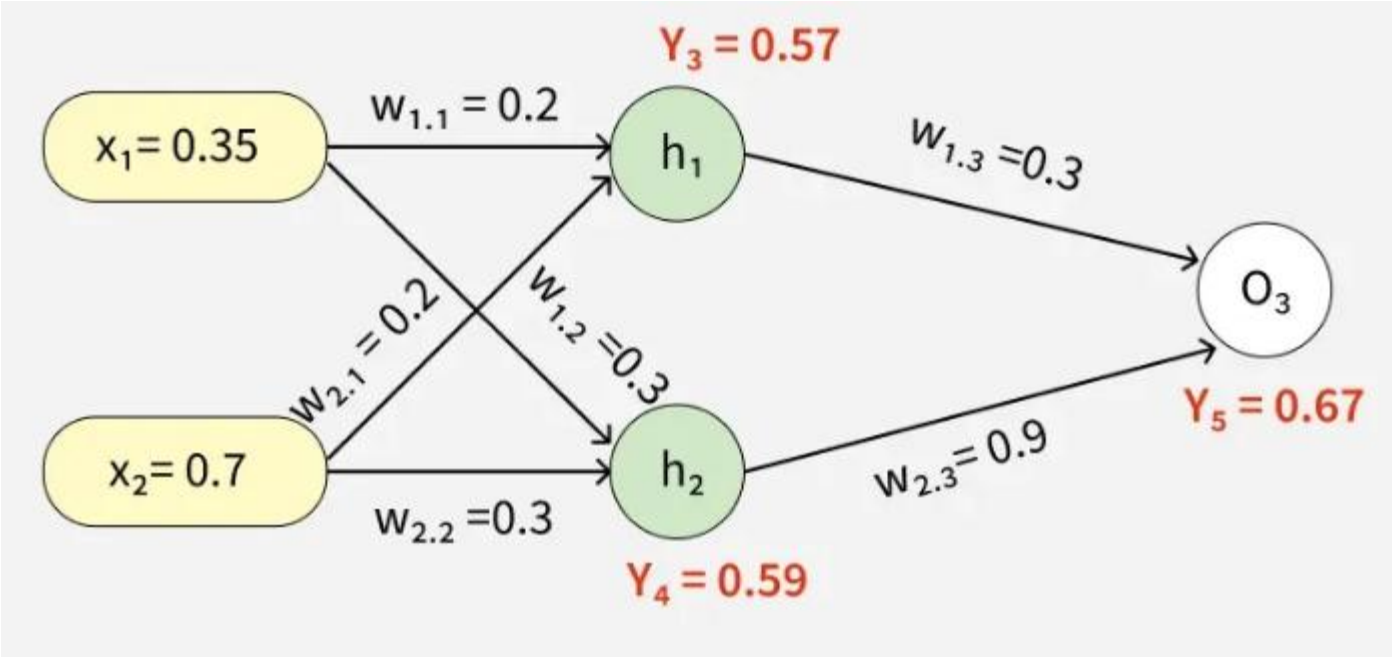
$$
\begin{aligned}
a_1 &= (w_{1,1}x_1) + (w_{2,1}x_2) \\
&= (0.2 * 0.35) + (0.2 * 0.7) \\
&= 0.21
\end{aligned}
$$

Once we calculated the a_1 value, we can now proceed to find the y_3 value:

$$
\begin{aligned}
y_j &= F(a_j) = \frac{1}{1+e^{-a_1}} \\
y_3 &= F(0.21) = \frac{1}{1+e^{-0.21}} \\
y_3 &= 0.56
\end{aligned}
$$

Similarly find the values of y_4 at $\mathbf{h_2}$ and y_5 at O_3

$$
\begin{aligned}
a_2 &= (w_{1,2} * x_1) + (w_{2,2} * x_2) = (0.3 * 0.35) + (0.3 * 0.7) = 0.315 \\
y_4 &= F(0.315) = \frac{1}{1+e^{-0.315}} \\
a3 &= (w_{1,3} * y_3) + (w_{2,3} * y_4) = (0.3 * 0.57) + (0.9 * 0.59) = 0.702 \\
y_5 &= F(0.702) = \frac{1}{1+e^{-0.702}} = 0.67
\end{aligned}
$$



4. Error Calculation

Our actual output is 0.5 but we obtained 0.67. To calculate the error we can use the below formula:

$$Error_j = y_{target} - y_5$$

$$\Rightarrow 0.5 - 0.67 = -0.17$$

Using this error value we will be backpropagating.

Back Propagation

1. Calculating Gradients

The change in each weight is calculated as:

$$\Delta w_{ij} = \eta \times \delta_j \times O_j$$

Where:

- δ_j is the error term for each unit,
- η is the learning rate.

2. Output Unit Error

For O3:

$$\delta_5 = y_5(1 - y_5)(y_{target} - y_5)$$
$$= 0.67(1 - 0.67)(-0.17) = -0.0376$$

3. Hidden Unit Error

For h1:

$$\delta_3 = y_3(1 - y_3)(w_{1,3} \times \delta_5)$$

$$= 0.56(1 - 0.56)(0.3 \times -0.0376) = -0.0027$$

For h2:

$$\delta_4 = y_4(1 - y_4)(w_{2,3} \times \delta_5)$$

$$= 0.59(1 - 0.59)(0.9 \times -0.0376) = -0.00819$$

4. Weight Updates

For the weights from hidden to output layer:

$$\Delta w_{2,3} = 1 \times (-0.0376) \times 0.59 = -0.022184$$

New weight:

$$w_{2,3}(\text{new}) = -0.022184 + 0.9 = 0.877816$$

For weights from input to hidden layer:

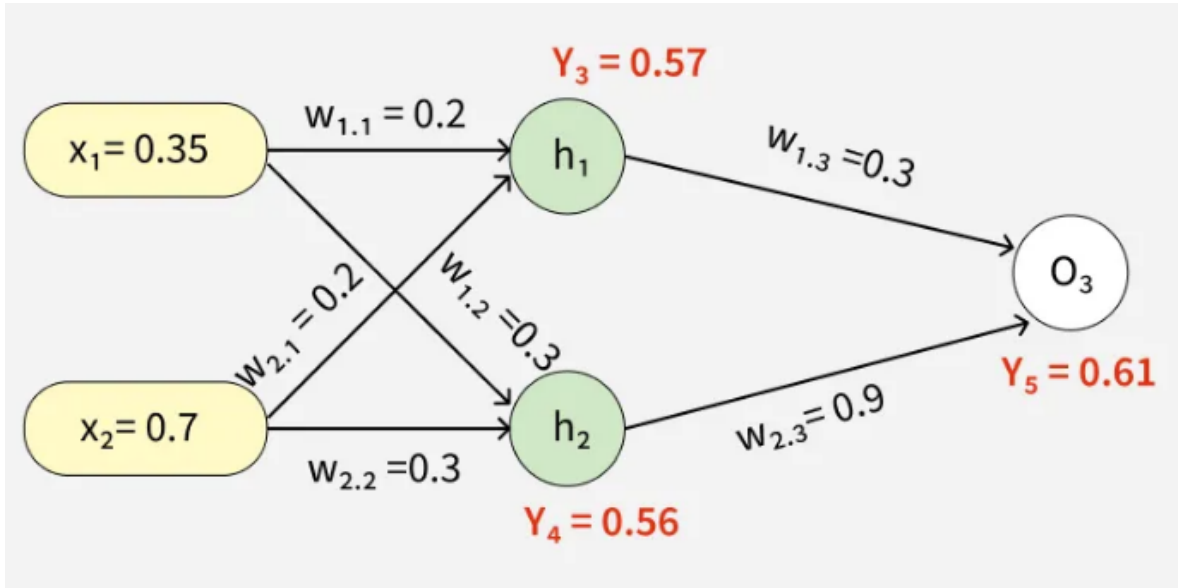
$$\Delta w_{1,1} = 1 \times (-0.0027) \times 0.35 = -0.000945$$

New weight:

$$w_{1,1}(\text{new}) = -0.000945 + 0.2 = 0.199055$$

Similarly other weights are updated:

- $w_{1,2}(\text{new}) = 0.273225$
- $w_{1,3}(\text{new}) = 0.086615$
- $w_{2,1}(\text{new}) = 0.269445$
- $w_{2,2}(\text{new}) = 0.18534$



After updating the weights the forward pass is repeated hence giving:

- $y_3 = 0.57$
- $y_4 = 0.56$
- $y_5 = 0.61$

Since $y_5 = 0.61$ is still not the target output the process of calculating the error and backpropagating continues until the desired output is reached.

This process demonstrates how Back Propagation iteratively updates weights by minimizing errors until the network accurately predicts the output.

$$Error = y_{target} - y_5$$

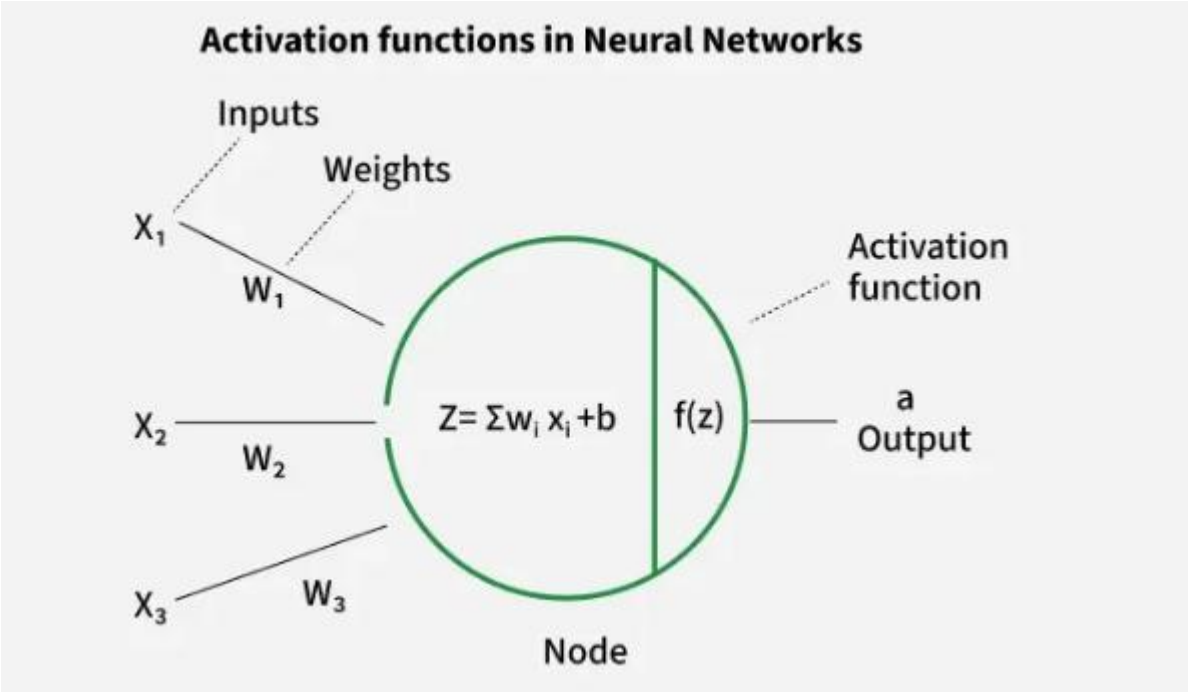
$$= 0.5 - 0.61 = -0.11$$

This process is said to be continued until the actual output is gained by the neural network.

Activation Functions in Neural Networks

Activation Functions in Neural Networks

An activation function is applied to the weighted sum of inputs before producing the final output of a neuron. It introduces non-linearity, allowing the network to learn complex patterns.



Activation Functions in Neural Networks

- *Applied after the weighted sum of inputs*
- *Introduces non-linearity into the model*
- *Enables learning of complex data patterns*
- *Without it, the network behaves like a linear model*

Importance of Non-Linearity

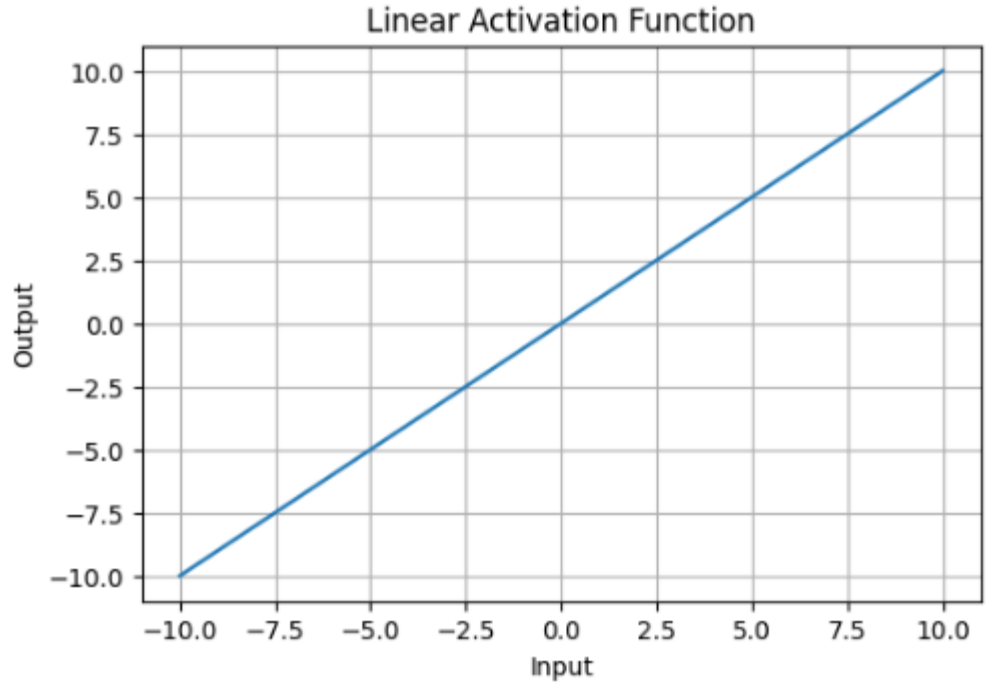
- *Real-world data is rarely linearly separable.*
- *Non-linear functions allow neural networks to form curved decision boundaries, making them capable of handling complex patterns (e.g., classifying apples vs. bananas under varying colors and shapes).*
- *They ensure networks can model advanced problems like image recognition, NLP and speech processing.*

Types of Activation Functions in Deep Learning

1. Linear Activation Function

Linear Activation Function resembles straight line define by y=x. No matter how many layers the neural network contains if they all use linear activation functions the output is a linear combination of the input.

- *The range of the output spans from (−∞ to +∞).*
- *Output is a linear combination of inputs*
- *Using it in all layers makes the network behave like a linear model*
- *Limits the ability to learn complex patterns*
- *Commonly used in the output layer for regression tasks*
- *Often combined with non-linear functions in hidden layers for better learning*



2. Non-Linear Activation Functions

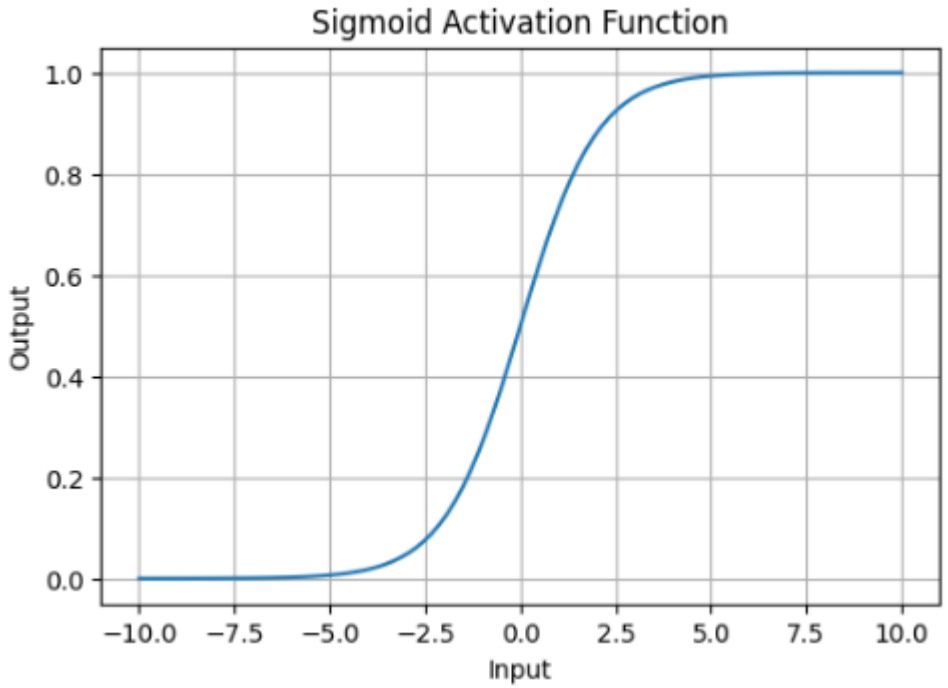
1. Sigmoid Function

Sigmoid Activation Function is characterized by 'S' shape. It is mathematically defined as

$$A = \frac{1}{1+e^{-x}}$$

This formula ensures a smooth and continuous output that is essential for gradient-based optimization methods.

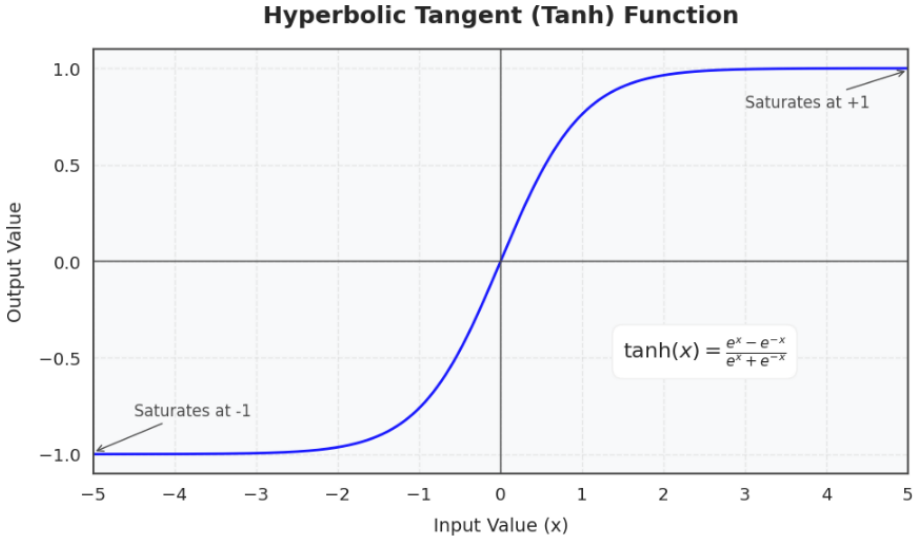
- *It allows neural networks to handle and model complex patterns that linear equations cannot.*
- *The output ranges between 0 and 1, hence useful for binary classification.*
- *The function exhibits a steep gradient when x values are between -2 and 2. This sensitivity means that small changes in input x can cause significant changes in output y which is critical during the training process.*



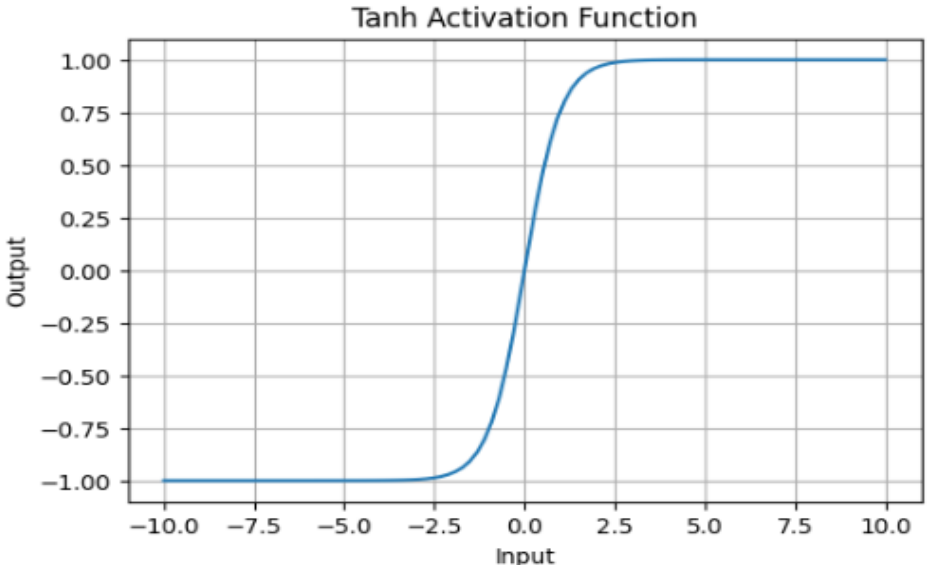
2. Tanh Activation Function

Tanh function (hyperbolic tangent function) is a shifted version of the sigmoid, allowing it to stretch across the y-axis. It is defined as:

$$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1.$$



- *Outputs values from -1 to +1.*
- *Enables modeling of complex data patterns.*
- *Commonly used in hidden layers due to its zero-centered output, facilitating easier learning for subsequent layers.*



https://www.geeksforgeeks.org/machine-learning/activation-functions-neural-networks/

3. ReLU(Rectified Linear Unit)Function

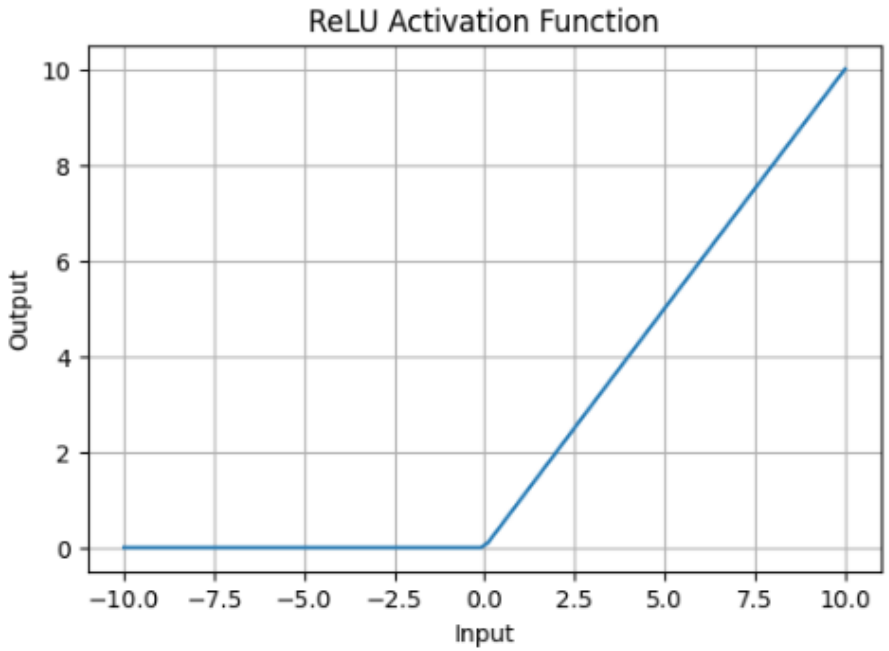
ReLU activation is defined by $A(x)=\max(0,x)$, this means that if the input x is positive, ReLU returns x, if the input is negative, it returns 0.

- *Value Range is$[0,\infty)$, meaning the function only outputs non-negative values.*
- *Introduces non-linearity, enabling learning of complex patterns*
- *Computationally efficient due to simple operations*
- *Activates only positive neurons, making the network sparse and efficient*
- *Commonly used in hidden layers for faster training and better performance*

4. Leaky ReLU

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

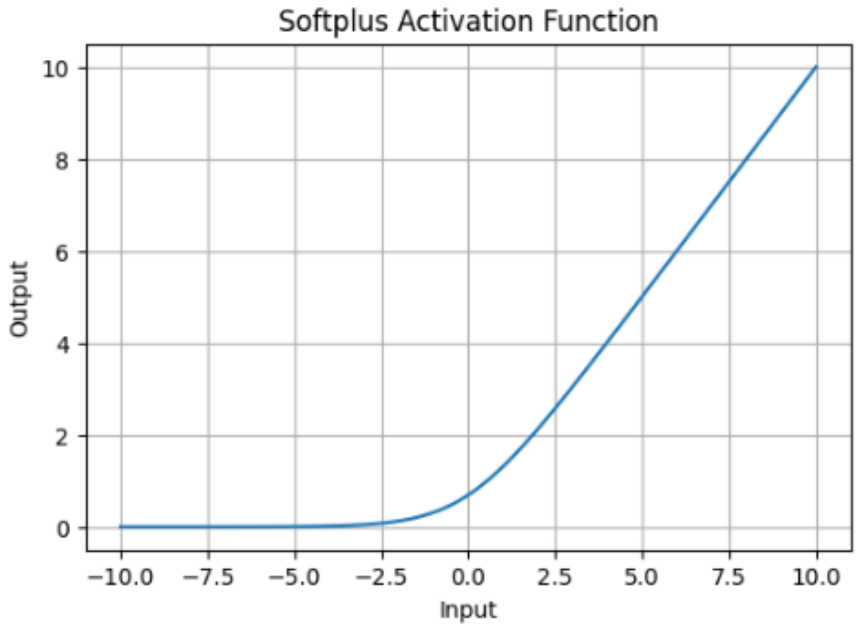
- Leaky ReLU is similar to ReLU but allows a small negative slope (α, e.g., 0.01) instead of zero.
- Solves the "dying ReLU" problem, where neurons get stuck with zero outputs.
- Range: $(-\infty, \infty)$.
- Preferred in some cases for better gradient flow.



5. SoftPlus Function

Softplus function is defined mathematically as: $A(x)=\log(1+ex)$. It is similar to ReLU but avoids sharp transitions by being fully differentiable.

- *The Softplus function is non-linear.*
- *The function outputs values in the range $(0,\infty)(0,\infty)$, similar to ReLU, but without the hard zero threshold that ReLU has.*
- *Softplus is a smooth, continuous function, meaning it avoids the sharp discontinuities of ReLU which can sometimes lead to problems during optimization.*



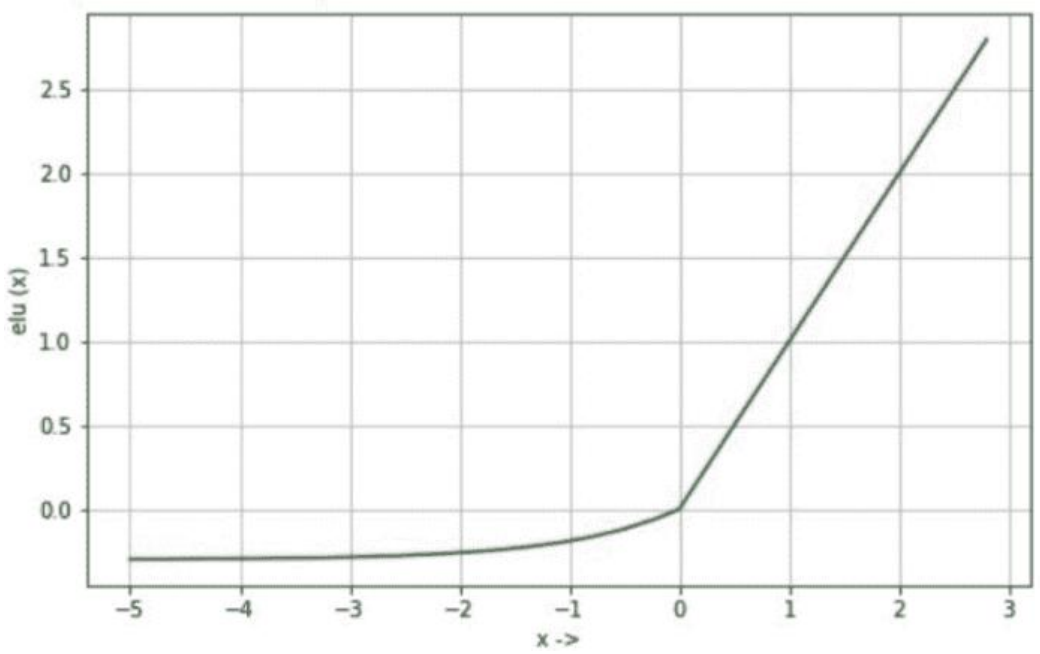
3. Exponential Linear Units

1. ELU (Exponential Linear Unit) Function

ELU (Exponential Linear Unit) is a non-linear activation function that improves learning speed and helps reduce the vanishing gradient problem. It behaves like ReLU for positive inputs but allows smooth negative values.

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

- Output range is $(-\alpha, \infty)$
- Introduces non-linearity for learning complex patterns
- Allows negative outputs, helping maintain zero-centered activations
- Smooth and differentiable, supporting stable training



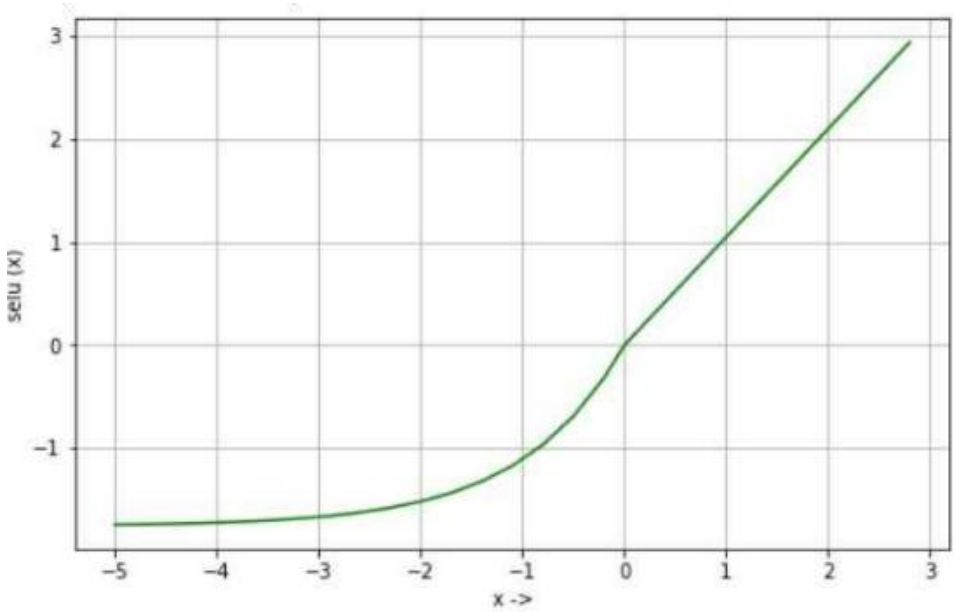
2. SELU (Scaled Exponential Linear Unit) Function

SELU is a scaled version of ELU designed for self-normalizing neural networks, helping maintain stable activations during training.

$$f(x) = \lambda \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

where $\lambda \approx 1.05$ (scaling factor) and $\alpha \approx 1.67$

- *Output range is$(-\lambda\alpha, \infty)(-\lambda\alpha, \infty)$*
- *Maintains near zero mean and unit variance (self-normalizing)*
- *Helps prevent vanishing and exploding gradients*
- *Works well in deep fully connected networks*
- *Can reduce the need for batch normalization in some cases*



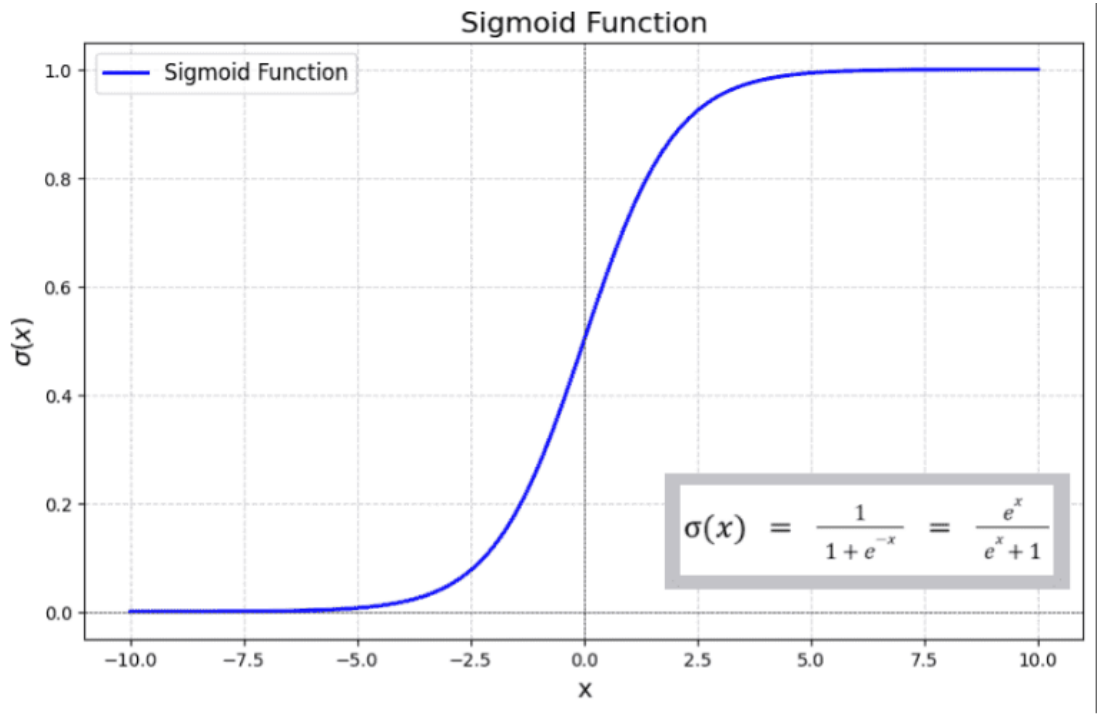
4. Output Layer Activation Functions

1. Sigmoid Activation Function

Sigmoid function produces an S-shaped curve and maps input values into a probability-like range between 0 and 1 and is used to find the final output of the neural network for binary classification problems. It is defined as:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

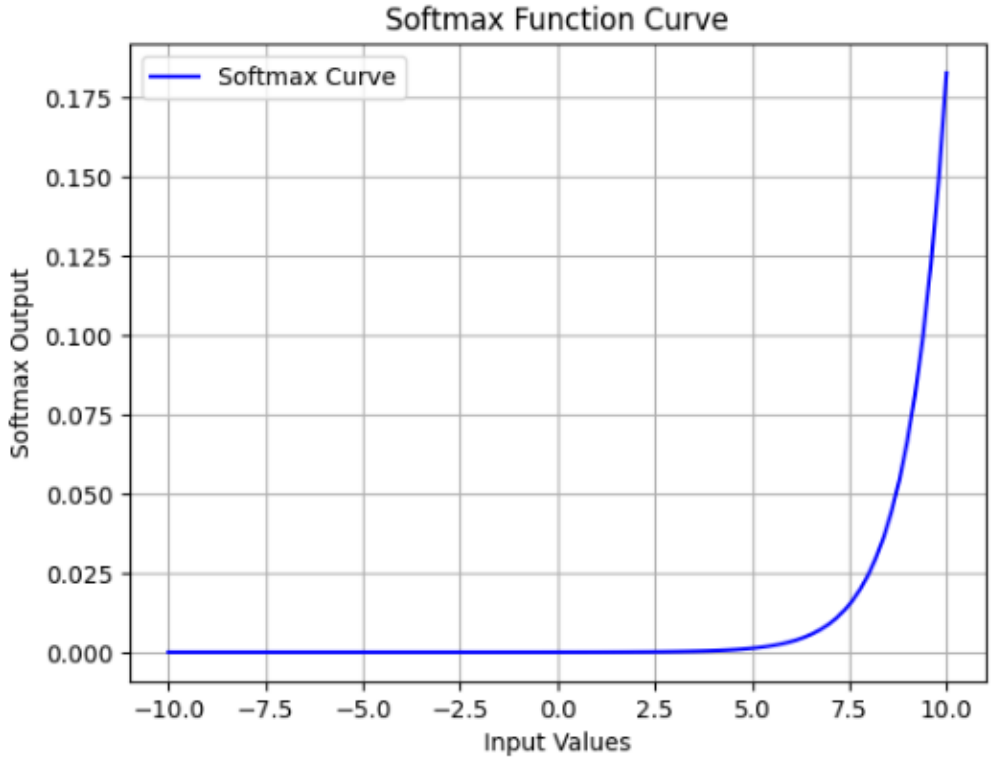
- *Output range is (0,1)*
- *Produces probability-like outputs*
- *Commonly used in the output layer for binary classification*
- *Smooth and differentiable, useful for gradient-based learning*



2. Softmax Function

Softmax function is used for multi-class classification and converts raw output scores into probabilities for each class.

- Transforms outputs into values between 0 and 1
- Ensures all probabilities sum to 1
- Highlights the most likely class among multiple options
- Commonly used in the output layer for multi-class classification
- Helps interpret model outputs as probabilities



Impact of Activation Functions on Model Performance

Activation functions play a key role in how efficiently a neural network learns and performs across different tasks.

1.ReLU helps in faster training by avoiding the vanishing gradient problem, while Sigmoid and Tanh can slow down convergence in deep networks.

2.ReLU maintains better gradient flow, allowing deeper layers to learn effectively, whereas Sigmoid may produce very small gradients.

3.Softmax enables handling of multi-class classification problems, while functions like ReLU or Leaky ReLU are commonly used in hidden layers for efficient learning.

Optimization Rule in Deep Neural Networks

Medium

Optimization Deep Neural Networks

In machine learning, optimizers and loss functions are two fundamental components that help improve a model’s performance.

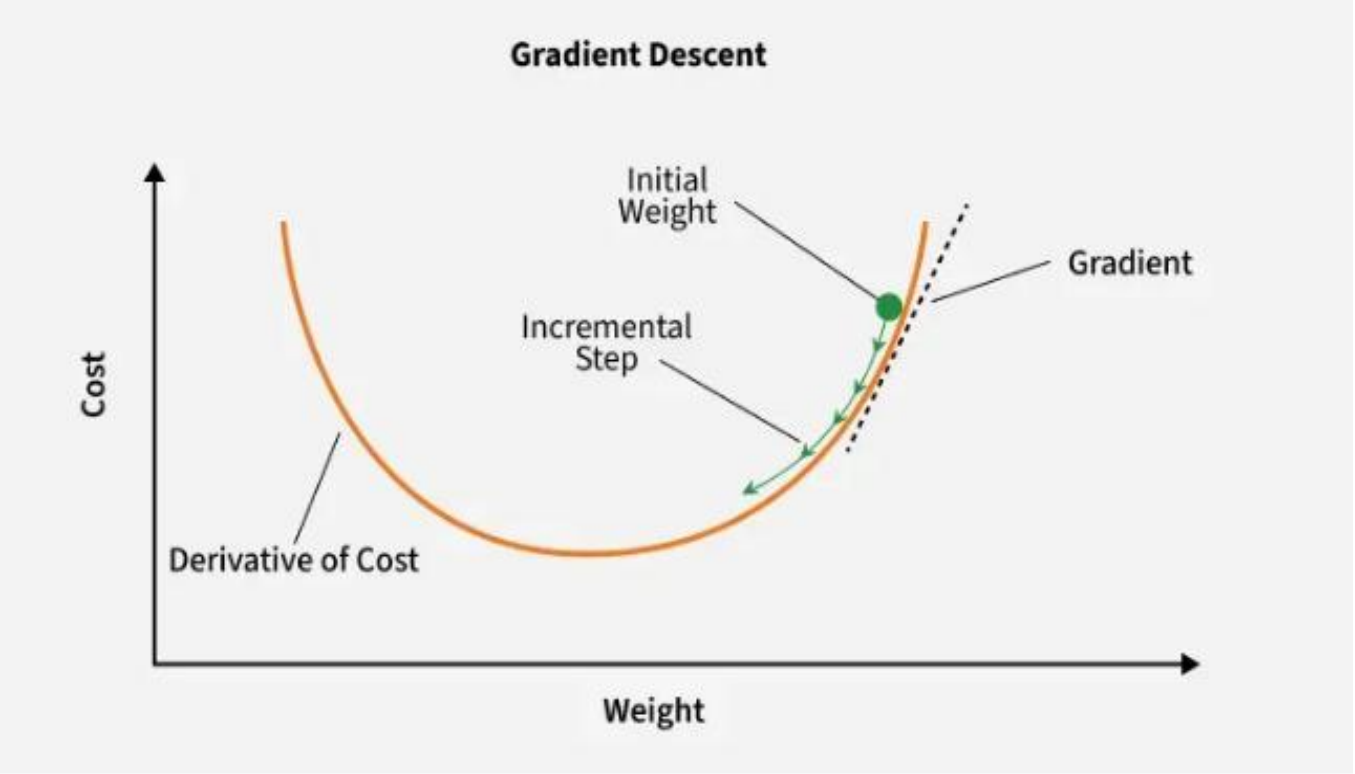
•A loss function evaluates a model's effectiveness by computing the difference between expected and actual outputs. Common loss functions include log loss, hinge loss and mean square loss.

•An optimizer improves the model by adjusting its parameters (weights and biases) to minimize the loss function value. Examples include RMSProp, ADAM and SGD (Stochastic Gradient Descent).

The optimizer’s role is to find the best combination of weights and biases that leads to the most accurate predictions.

Gradient Descent

Gradient Descent is a popular optimization method for training machine learning models. It works by iteratively adjusting the model parameters in the direction that minimizes the loss function



Key Steps in Gradient Descent

***1.Initialize parameters**: Randomly initialize the model parameters.*

***2.Compute the gradient**: Calculate the gradient (derivative) of the loss function with respect to the parameters.*

***3.Update parameters**: Adjust the parameters by moving in the opposite direction of the gradient, scaled by the learning rate.*

Formula:

General Update Rule: $w = w - \alpha \frac{\partial L}{\partial w}$

$b = b - \alpha \frac{\partial L}{\partial b}$

where

$\alpha : learning\ rate$

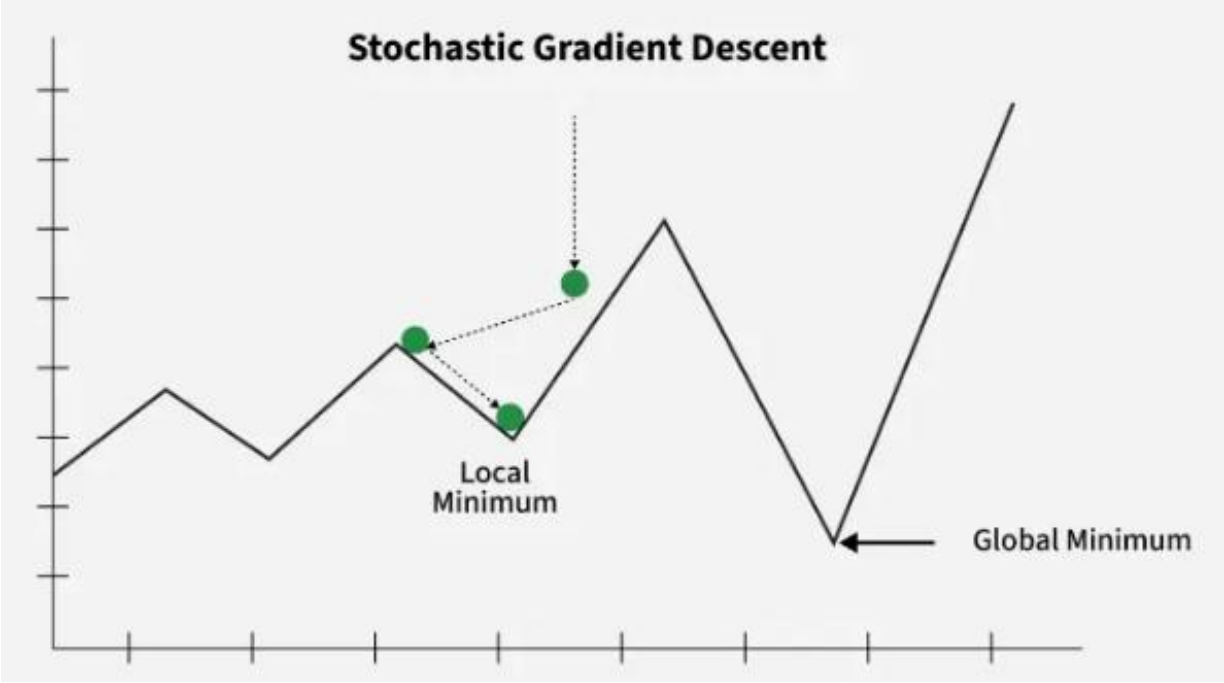
$L : loss\ function$

$\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b} : gradients\ of\ loss$

Variants of Gradient Descent

1. Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) updates the model parameters after each training example, making it more efficient for large datasets compared to traditional Gradient Descent, which uses the entire dataset for each update.



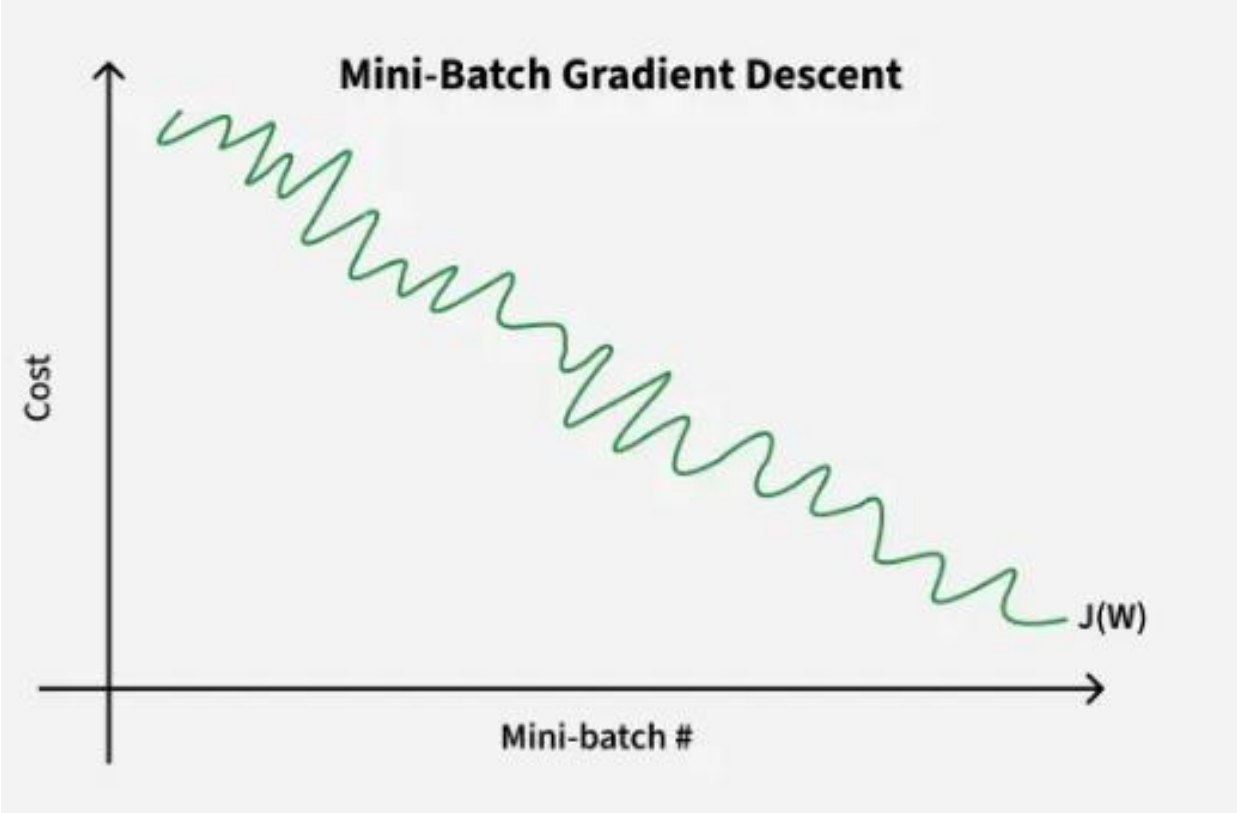
Steps:

1.Select a training example.
2.Compute the gradient of the loss function.
3.Update the model parameters.

Advantages: Requires less memory and may find new minima.
Disadvantages: Noisier, requiring more iterations to converge.

2. Mini Batch Gradient Descent

Mini-batch gradient descent consists of a predetermined number of training examples, smaller than the full dataset. This approach combines the advantages of the previously mentioned variants.



In one epoch, following the creation of fixed-size mini-batches, we execute the following steps:
1.Select a mini-batch.
2.Compute the mean gradient of the mini-batch.
3.Apply the mean gradient obtained in step 2 to update the model's weights.
4.Repeat steps 1 to 2 for all the mini-batches that have been created.

Advantages: Requires medium amount of memory and less time required to converge when compared to SGD
Disadvantage: May get stuck at local minima

3. SGD with Momentum

Momentum helps accelerate convergence by smoothing out the noisy gradients of SGD, thus reducing fluctuations and improving the speed of convergence.

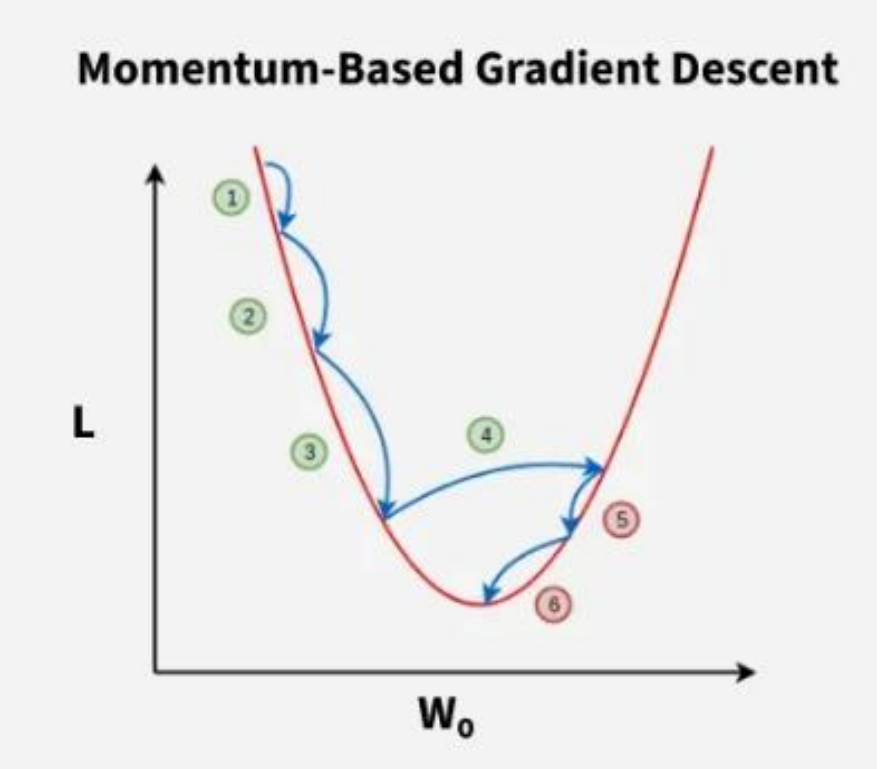
$$v_{(t+1)} = \beta * v_t + (1 - \beta) * \nabla J(\theta_t)$$

Where:

- v_t is the momentum at time t.
- β is the momentum coefficient.
- $\nabla J(\theta_t)$ is the gradient at time t.

Then, the model parameters are updated using:

$$\theta_{(t+1)} = \theta_t - \alpha * v_{(t+1)}$$



Advantages: Mitigates oscillations, reduces variance and faster convergence.
Disadvantages: Requires tuning the momentum coefficient β.

Advanced Optimizers

1. AdaGrad

AdaGrad adapts the learning rate for each parameter based on the historical gradient information. The learning rate decreases over time, making AdaGrad effective for sparse features.

$$\theta_{(t+1)} = \theta_t - \frac{\alpha}{\sqrt{G_t + \varepsilon}} * \nabla J(\theta_t)$$

Where:

- G_t is the sum of squared gradients.
- ε is a small constant to avoid division by zero.

Advantages: Adapts the learning rate, improving training efficiency.
Disadvantages: Learning rate decays too quickly, causing slow convergence.

2. RMSProp

RMSProp improves upon AdaGrad by introducing a decay factor to prevent the learning rate from decreasing too rapidly.

$$E[g^2]_t = \gamma * E[g^2]_{(t-1)} + (1 - \gamma) * (\nabla J(\theta_t))^2$$

$$\theta_{(t+1)} = \theta_t - \frac{\alpha}{\sqrt{E[g^2]_t + \varepsilon}} * \nabla J(\theta_t)$$

Where:

- γ is the decay rate.
- $E[g^2]_t$ is the exponentially moving average of squared gradients.

Advantages: Prevents excessive decay of learning rates.
Disadvantages: Computationally expensive due to the additional parameter.

3. Adam (Adaptive Moment Estimation)

Adam combines the advantages of Momentum and RMSProp. It uses both the first moment (mean) and second moment (variance) of gradients to adapt the learning rate for each parameter.

1. Update the first moment: $m_t = \beta_1 * m_{(t-1)} + (1 - \beta_1) * \nabla J(\theta_t)$
2. Update the second moment: $v_t = \beta_2 * v_{(t-1)} + (1 - \beta_2) * (\nabla J(\theta_t))^2$
3. Bias correction: $\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$
4. Update parameters: $\theta_{(t+1)} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t + \varepsilon}} * \hat{m}_t$

Where:

- β_1 and β_2 are the decay rates for the first and second moments.
- ε is a small constant to prevent division by zero.

Advantages: Fast convergence.
Disadvantages: Requires significant memory due to the need to store first and second moment estimates.

Comparison of Optimizers

Optimizer	Advantages	Disadvantages
SGD	Simple, easy to implement	Slow convergence, requires tuning
Mini-Batch SGD	Faster than SGD	Computationally expensive, stuck in local minima
SGD with Momentum	Faster convergence, reduces noise	Requires careful tuning of β
AdaGrad	Adaptive learning rates	Decays too fast, slow convergence
RMSProp	Prevents fast decay of learning rates	Computationally expensive
Adam	Fast, combines momentum and RMSProp	Memory-intensive, computationally expensive

Each optimizer has its own strengths and weaknesses. The choice of optimizer depends on the specific problem, dataset characteristics and the computational resources available. Adam is often the default choice due to its robust performance, but each situation may call for a different optimizer to achieve optimal results.

Thank you